

MODELO CONCEPTUAL DE LA RED SOCIAL ESTUDIANTIL PASCUALINA.

PRESENTADO POR:

-NATHALY RIVAS ARISTIZABAL.

CURSO:

BASES DE DATOS

TUTORA:

ORALIA CORTÉS CARVAJAL.

G100

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO.

2026

1. Análisis de necesidades y acciones del usuario.

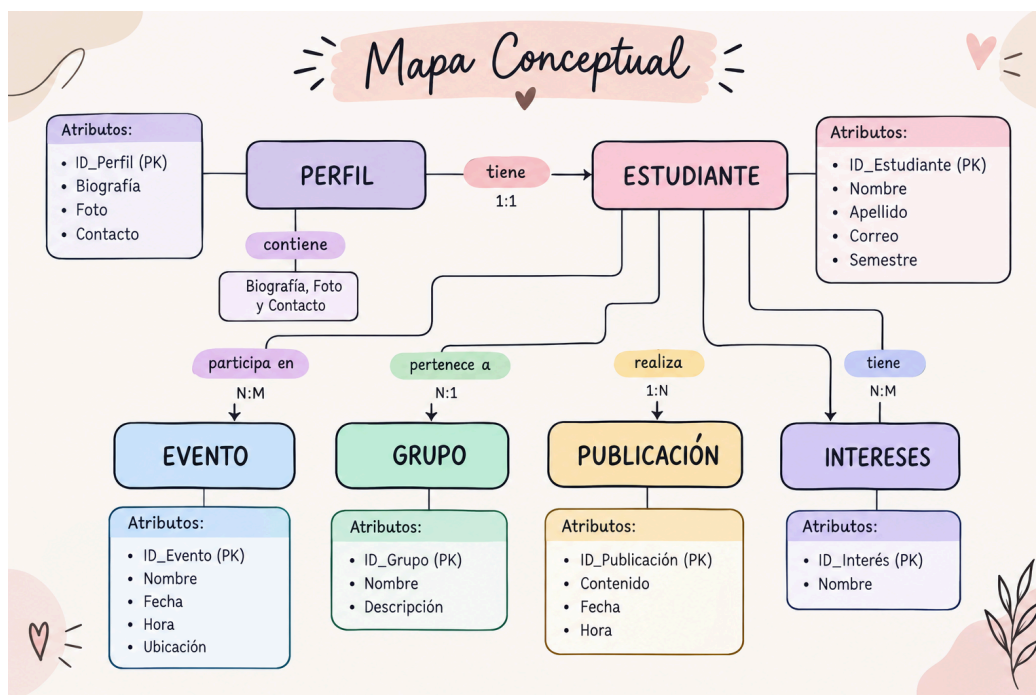
Tras revisar los requerimientos de la Red Social Estudiantil Pascualina, identifique las siguientes acciones principales que los estudiantes deben realizar en la plataforma:

- Gestión de Perfiles: Crear y actualizar perfiles con información académica, área de estudio e intereses.
- Interacción y Contenidos: Publicar noticias, preguntas sobre tareas y recursos útiles en tiempo real.
- Trabajo Colaborativo: Crear y unirse a grupos de estudio, clubes de interés o equipos para hackatones.
- Gestión de Eventos: Agendar y coordinar talleres, reuniones de estudio y actividades académicas o sociales.

2.Lluvia de ideas (ENTIDADES)

- Estudiante
- Grupo
- Evento
- Perfil
- Intereses
- Publicación

Mapa de conceptos



3. Modelado lógico y diccionario de datos.

Para que la base de datos de la Red Social Pascualina funcione de manera ordenada y sin repetir información innecesaria, organizamos los datos aplicando el proceso de normalización paso a paso a partir de las necesidades del proyecto.

*Estado inicial (0FN) En la etapa inicial, todos los datos del estudiante estaban guardados en una sola lista desordenada. En un mismo registro se mezclaban sus datos personales, su programa académico, sus intereses y sus publicaciones. Esto generaba duplicación de datos y listas repetidas en una sola celda.

*Primera Forma Normal (1FN) Para cumplir con esta regla se eliminaron las listas repetidas y se aseguró que cada casilla contenga un solo valor. La lista de intereses se separó en una tabla independiente, y lo mismo se hizo con las publicaciones y los eventos, asignándole a cada elemento un número de identificación único como clave primaria.

*Segunda Forma Normal (2FN) Esta regla busca que todos los datos de una tabla dependan únicamente de su identificador principal. En el caso de las tablas que conectan dos entidades como la tabla que une a los estudiantes con los grupos, los datos como la fecha de ingreso y el rol guardan relación con esa combinación específica. Además, la información del perfil del estudiante se integró con sus datos principales para evitar duplicar identificadores.

*Tercera Forma Normal (3FN) En esta última fase se eliminaron los datos que dependían de otros atributos que no eran la clave principal. Por ejemplo, en la información académica se detectó que la facultad depende de la carrera y no directamente del estudiante. Por esta razón se creó la tabla independiente de carreras con su propio identificador.

*Diccionario de datos

Tabla de estudiantes

Nombre del Atributo	Tipo de Dato y Longitud	Descripción	Justificación de la Elección
id_estudiante	INT	Identificador único del estudiante (PK)	Entero eficiente para búsquedas rápidas e indexación.
nombre	VARCHAR(50)	Nombre del estudiante	Espacio adecuado para nombres comunes sin

			malgastar almacenamiento.
apellido	VARCHAR(50)	Apellidos del usuario	Tamaño estándar para almacenar apellidos.
correo	VARCHAR(100)	Correo institucional único	Permite autenticar al usuario y garantiza que no existan duplicados.
semestre	INT	Semestre actual	Tipo numérico entero que facilita filtrar por nivel académico.
biografia	TEXT	Descripción personal	Permite escribir texto libre sin la limitación de un límite corto.
foto	VARCHAR(255)	Enlace o ruta de la foto de perfil	Tamaño adecuado para guardar direcciones web o rutas de archivos.
contacto	VARCHAR(50)	Teléfono o red social secundaria	Permite guardar un medio de contacto alternativo.
id_carrera	INT	Referencia a la carrera (FK)	Clave foránea que relaciona al estudiante con su programa de estudio.

Tabla carreras.

Nombre del Atributo	Tipo de Dato y Longitud	Descripción	Justificación de la Elección
id_carrera	INT	Identificador único de la carrera (PK)	Clave primaria entera autonumerada.

nombre_carrera	VARCHAR(100)	Nombre del programa académico	Espacio suficiente para nombres de títulos largos.
facultad	VARCHAR(100)	Facultad a la que pertenec	Permite normalizar la dependencia transitiva (3FN).

Tabla publicaciones

Nombre del Atributo	Tipo de Dato y Longitud	Descripción	Justificación de la Elección
id_publicacion	INT	Identificador de la publicación (PK)	Clave primaria única para cada mensaje.
contenido	TEXT	Texto del mensaje o recurso compartido	Permite publicaciones extensas sin restricciones cortas.
fecha	DATE	Fecha de creación del post	Permite ordenar las publicaciones en el muro de forma cronológica.
hora	TIME	Hora exacta de creación	Otorga precisión para el ordenamiento temporal de los mensajes.
id_estudiante	INT	Autor de la publicación (FK)	Vincula el contenido de manera directa con el estudiante que lo escribió.

Tabla grupos

Nombre del Atributo	Tipo de Dato y Longitud	Descripción	Justificación de la Elección
id_grupo	INT	Identificador único del grupo (PK	Clave primaria entera.
nombre	VARCHAR(100)	Nombre de la comunidad o grupo	Identifica de forma clara la agrupación o club.
descripcion	TEXT	Propósito y reglas del grupo	Permite detallar los objetivos y la información de la comunidad.

Tabla eventos

Nombre del Atributo	Tipo de Dato y Longitud	Descripción	Justificación de la Elección
id_evento	INT	Identificador único del evento (PK)	Clave primaria para organizar reuniones o talleres.
nombre	VARCHAR(100)	Título de la actividad	Nombre descriptivo del evento programado.
fecha	DATE	Día en que se realizará	Almacena la fecha de la convocatoria.
hora	TIME	Horario de inicio	Almacena la hora fijada para el encuentro.
ubicacion	VARCHAR(150)	Lugar físico o enlace virtual	Espacio donde se desarrollará la actividad.

Tabla intereses

Nombre del Atributo	Tipo de Dato y Longitud	Descripción	Justificación de la Elección
id_interes	INT	Identificador del interés (PK)	Clave primaria entera para catalogación.
nombre	VARCHAR(50)	Etiqueta del gusto o habilidad	Cadena corta para categorizar temas como Programación o Deportes.

4. Explicación sobre elección de nombres y conversiones de entidad a tabla.

Para transformar las entidades conceptuales en las tablas finales de mi base de datos, apliqué y decidí los siguientes criterios de diseño:

Escribí todos los nombres en minúsculas y con guion bajo (snake_case): Elegí nombrar todas las tablas y atributos en minúsculas separadas por guiones bajos, como por ejemplo id_estudiante, nombre_carrera y fecha_publicacion. Tomé esta decisión para evitar errores de compatibilidad entre mayúsculas y minúsculas al ejecutar consultas en motores como PostgreSQL o MySQL.

Cambié los nombres de las tablas a plural: Como cada tabla almacena un conjunto de registros, convertí las entidades de singular a plural, como por ejemplo estudiantes, carreras, grupos, publicaciones, eventos e intereses.

Definé un identificador claro para cada tabla: Creé una clave primaria única para cada tabla utilizando el prefijo id_ seguido del nombre de la entidad, como por ejemplo id_grupo. Esto me facilitó conectar las tablas entre sí mediante claves foráneas.

Integré la entidad perfil en la tabla de estudiantes: Decidí no crear una tabla independiente para el perfil; en su lugar, guardé la biografía, la foto y el contacto directamente en la tabla estudiantes para simplificar la estructura y evitar consultas innecesarias.

* Tabla descriptiva para relaciones.

Nombre de la Relación	Tablas involucradas	Cardinalidad	Implementación lógica propuesta	Justificación de la implementación
Estudiante tiene perfil	estudiantes	1:1 (un estudiante tiene 1 perfil)	Se integraron los atributos de perfil (biografía, foto, contacto) directamente en la tabla estudiantes.	Para evitar crear una tabla adicional con relación uno a uno que sobrecargue las consultas, simplificando el acceso a los datos del usuario.
Estudiante pertenece a carrera	estudiantes carreras	1:n (una carrera tiene n estudiantes)	Se añade el atributo id_carrera de tipo int a la tabla estudiantes.	Permite asociar a cada estudiante con su programa académico sin duplicar la información de las facultades
Estudiante realiza publicación	estudiantes publicaciones	1:n (un estudiante tiene n publicaciones)	Se añade el atributo id_estudiante de tipo int a la tabla publicaciones.	Para que cada publicación esté vinculada únicamente a su autor, permitiendo consultar todas las publicaciones creadas por un estudiante.
Estudiante pertenece a grupo	estudiantes grupos	n:m (n estudiantes pertenecen a m grupos)	Se crea la tabla intermedia membresias_grupo con id_estudiante e id_grupo como claves foráneas.	Permite romper la relación muchos a muchos, logrando que un estudiante esté en varios grupos y que cada grupo tenga múltiples miembros.
Estudiante participa en	estudiantes eventos	n:m (n estudiantes)	Se crea la tabla intermedia	Permite registrar la

evento		participan en m eventos)	estudiantes_ev entos con id_estudiante e id_evento como claves foráneas.	asistencia e inscripción de múltiples estudiantes a diferentes actividades o talleres.
Estudiante tiene intereses	estudiantes intereses	n:m (un estudiante tiene m intereses)	Se crea la tabla intermedia estudiantes_int ereses con id_estudiante e id_interes como claves foráneas.	Evita guardar listas repetidas de texto en una sola casilla, asignando varios intereses a cada estudiante de forma organizada.

***Diagrama de modelo logico

Puse el diagrama de forma horizontal debido a que casi no se puede observar la informacion.

carreras			
id_carrera $\emptyset$	int		
nombre_carrera	varchar		
facultad	varchar		

grupos			
id_grupo $\emptyset$	int		
nombre	varchar		
descripcion	text		

eventos				
id_evento $\emptyset$	int			
nombre	varchar			
fecha	date			
hora	time			
ubicacion	varchar			

intereses		
id_interes $\emptyset$	int	
nombre	varchar	

estudiantes										
id_estudiante $\emptyset$	int									
nombre	varchar									
apellido	varchar									
correo	varchar									
semestre	int									
biografia	text									
foto	varchar									
contacto	varchar									
id_carrera $\emptyset$	int									

publicaciones					
id_publicacion $\emptyset$	int				
contenido	text				
fecha	date				
hora	time				
id_estudiante $\emptyset$	int				

estudiantes_grupos				
id_estudiante $\emptyset$	int			
id_grupo $\emptyset$	int			
fecha_ingreso	date			
rol	varchar			

estudiantes_eventos			
id_estudiante $\emptyset$	int		
id_evento $\emptyset$	int		
estado_asistencia	varchar		

estudiantes_intereses		
id_estudiante $\emptyset$	int	
id_interes $\emptyset$	int	

Conclusion: Durante el desarrollo de esta actividad comprendí la importancia de estructurar correctamente un modelo lógico de base de datos partiendo de un diseño conceptual. El proceso de normalización hasta la tercera forma normal me permitió identificar cómo eliminar la redundancia de información y garantizar la integridad de los datos, especialmente al unificar los atributos del perfil dentro de la entidad principal de estudiantes para simplificar la arquitectura. Asimismo, el reto más significativo consistió en la correcta transformación de las relaciones de muchos a muchos mediante la creación de tablas intermedias con claves compuestas, lo que aseguró que la estructura sea completamente funcional e implementable en un sistema de gestión de bases de datos real. Esta experiencia fortaleció mi capacidad para analizar requerimientos de software y traducirlos en diagramas técnicos coherentes y normalizados.